



IRBID NATIONAL
UNIVERSITY

جامعة إربد الأهلية

OPERATING SYSTEMS

A CORE OF COMPUTING. A FOUNDATION FOR INNOVATION.



PREPARED BY:

- Basema Thabet Odat : 202310822
- Diala Belal Zaki Awwad : 202311057
- Ghufraan Marai Dorgham : 202311523
- Sara Luay Ibrahim Bataineh : 202311093
- Sedra Bashar Emad Addin Alsaqqal : 20230832



COURSE:

Operating Systems



INSTRUCTOR:

Dr. Mohammad Alomar

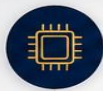


UNIVERSITY:

Irbid National University



1. INTRODUCTION



Modern operating systems are responsible for managing hardware resources efficiently while ensuring that multiple processes can execute smoothly and fairly. One of the most critical responsibilities of the operating system is **CPU scheduling**. CPU scheduling determines which process will use the CPU at a specific moment and directly affects the overall system performance, responsiveness, throughput, and user experience.



In **multitasking environments**, many processes compete simultaneously for CPU access. Without an efficient scheduling mechanism, the CPU may become inefficiently utilized, processes may experience excessive waiting times, and system responsiveness may degrade significantly. For this reason, operating systems implement different **scheduling algorithms**, where each algorithm follows a unique strategy for process selection and execution.



This project focuses on designing and implementing a complete **CPU Scheduling Simulator** using **Python**. The simulator implements four fundamental scheduling algorithms:



The system evaluates the behavior and efficiency of each scheduling algorithm under multiple workload scenarios. Additionally, the simulator generates detailed **Gantt charts** and **performance comparison graphs** to visualize execution order and analyze scheduling efficiency.



The primary purpose of this project is not only to implement scheduling algorithms but also to perform analytical comparisons between them using important performance metrics such as:



The project demonstrates how different scheduling approaches behave under various workloads and highlights the strengths and limitations of each algorithm in practical operating system environments.



THE SIMULATOR IMPLEMENTS FOUR FUNDAMENTAL SCHEDULING ALGORITHMS:



**First Come
First Serve
(FCFS)**



**Shortest
Job First
(SJF)**



**Round
Robin
(RR)**



**Priority
Scheduling**

THE PRIMARY PURPOSE IS TO PERFORM ANALYTICAL COMPARISONS USING IMPORTANT PERFORMANCE METRICS:



**Average
Waiting Time**



**Average
Turnaround Time**



Fairness



Responsiveness

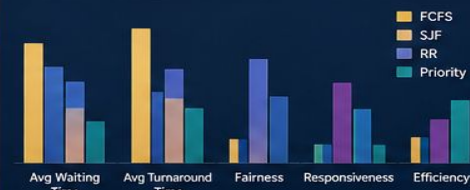


**Scheduling
Efficiency**

GANTT CHART EXAMPLE



PERFORMANCE COMPARISON



2. PROJECT OBJECTIVES

The main **objectives** of this project are:



01

To design and implement a **CPU Scheduling Simulator** using **Python**.



02

To **simulate** the execution behavior of multiple **scheduling algorithms**.



03

To calculate scheduling **performance metrics** accurately.



04

To generate graphical **Gantt chart visualizations** for process execution.



05

To **compare** algorithm **performance** under different scheduling scenarios.



06

To analyze the **efficiency, fairness, and responsiveness** of each scheduling technique.



07

To **identify** which algorithms **perform better** under specific workload conditions.



08

To provide **practical understanding** of CPU scheduling concepts in operating systems.



OBJECTIVES



3. CPU SCHEDULING ALGORITHMS

3.1 First Come First Serve (FCFS)

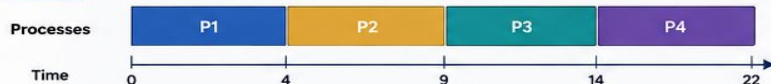
First Come First Serve is the simplest CPU scheduling algorithm. Processes are executed according to their arrival order in the ready queue. The process that arrives first receives CPU execution first.



WORKING PRINCIPLE

- Processes are inserted into the ready queue according to arrival time.
- The scheduler selects the earliest arriving process.
- Once a process starts execution, it continues until completion.

Example



ADVANTAGES

- Simple and easy to implement.
- Fair in terms of arrival order.
- Minimal scheduling overhead.



DISADVANTAGES

- Poor average waiting time.
- Suffers from the convoy effect.
- Long processes can delay short processes significantly.
- Weak responsiveness for interactive systems.



SUITABLE ENVIRONMENTS

FCFS is suitable for simple batch systems where fairness is more important than responsiveness.

3.2 Shortest Job First (SJF)

Shortest Job First selects the process with the smallest CPU burst time.



WORKING PRINCIPLE

- Among all ready processes, the scheduler selects the process with the shortest burst time.
- Short processes complete earlier, reducing average waiting time.

Example



ADVANTAGES

- Produces optimal average waiting time in many cases.
- Improves system throughput.
- Efficient for systems containing many short tasks.



DISADVANTAGES

- Possible starvation for long processes.
- Burst time prediction may not always be accurate.
- More complex than FCFS.



SUITABLE ENVIRONMENTS

SJF is highly efficient in systems where burst times can be estimated accurately.



3. CPU SCHEDULING ALGORITHMS

3.3 Round Robin (RR)



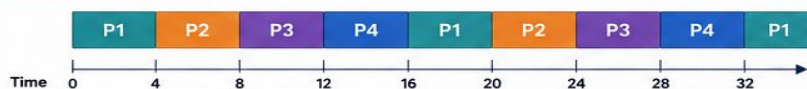
Round Robin scheduling allocates CPU time using fixed time slices called **quantum**.



WORKING PRINCIPLE

- Each process receives CPU execution for a limited time quantum.
- If execution is incomplete, the process returns to the end of the ready queue.
- CPU cycles continuously between processes.

Example (Quantum = 4)



ADVANTAGES

- High fairness.
- Excellent responsiveness.
- Suitable for time-sharing systems.
- Prevents process starvation.



DISADVANTAGES

- Performance depends heavily on quantum size.
- Excessive context switching may reduce efficiency.
- Larger turnaround times in some workloads.



SUITABLE ENVIRONMENTS

Round Robin is ideal for interactive and multi-user operating systems.



3.4 Priority Scheduling



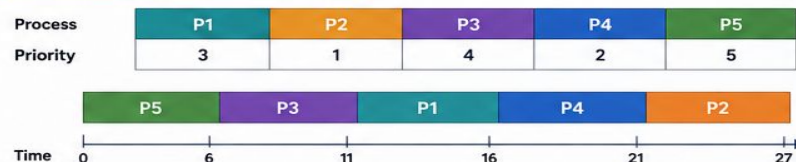
Priority Scheduling executes processes according to **priority values**.



WORKING PRINCIPLE

- Each process is assigned a priority.
- Higher priority processes execute before lower priority processes.
- Equal priorities may follow FCFS ordering.

Example (Higher number = Higher priority)



ADVANTAGES

- Important processes receive faster execution.
- Flexible scheduling behavior.
- Useful for real-time and critical systems.



DISADVANTAGES

- Starvation may occur for low-priority processes.
- Requires careful priority management.
- Fairness may decrease significantly.



SUITABLE ENVIRONMENTS

Priority Scheduling is commonly used in systems containing critical or time-sensitive processes.

High
Priority



Low
Priority



4. SYSTEM DESIGN AND METHODOLOGY



The simulator was designed to emulate realistic CPU scheduling behavior while maintaining **modularity** and **scalability**.

The system accepts multiple process inputs where each process contains:



Process ID



Arrival Time



Burst Time

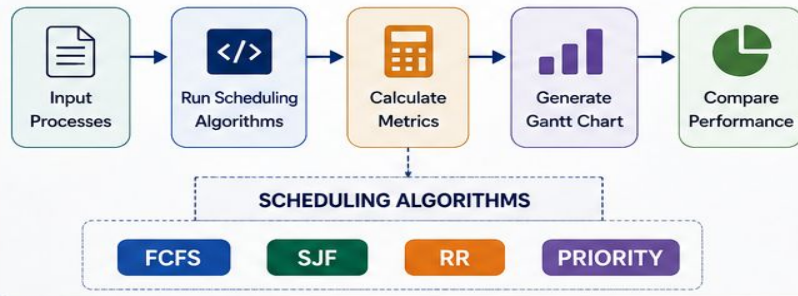


Priority



The simulator executes each scheduling algorithm independently and calculates **scheduling statistics** automatically.

SYSTEM WORKFLOW



4.1 PERFORMANCE METRICS



WAITING TIME (WT)

Waiting Time represents the amount of time a process spends waiting in the ready queue before execution.

$$WT = TAT - BT$$

Where:

- WT = Waiting Time
- TAT = Turnaround Time
- BT = Burst Time



TURNAROUND TIME (TAT)

Turnaround Time measures total completion duration from process arrival until execution completion.

$$TAT = CT - AT$$

Where:

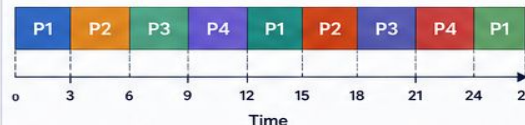
- CT = Completion Time
- AT = Arrival Time

4.2 GANTT CHART VISUALIZATION



The simulator generates graphical Gantt charts using **Matplotlib** to visualize:

Example Gantt Chart (Round Robin, Quantum = 3)



Process execution order



CPU allocation timeline



Scheduling behavior differences



These visualizations help analyze **fairness**, process **switching frequency**, and scheduling **efficiency**.

5. IMPLEMENTATION



The project was implemented using **Python** due to its simplicity, flexibility, and strong visualization libraries.



TECHNOLOGIES USED



Python 3



Matplotlib



Pandas



MAIN FUNCTIONALITIES

The simulator performs the following operations:



Process scheduling simulation



Waiting time calculation



Turnaround time calculation



Performance comparison



Gantt chart generation



Scenario-based workload testing

```
1 def schedule(processes):  
2     for p in processes:  
3         execute(p)  
4     return results
```



PROGRAM STRUCTURE

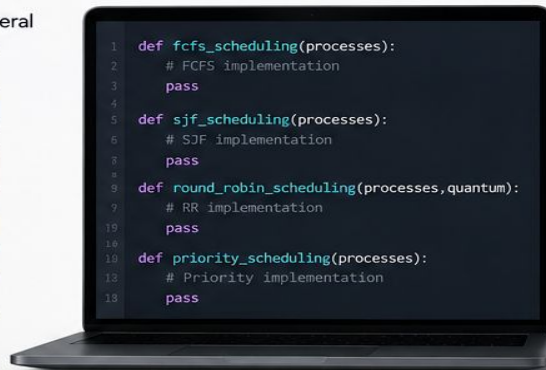
The implementation consists of several independent scheduling functions:

```
</> fcfs_scheduling()
```

```
</> sjf_scheduling()
```

```
</> round_robin_scheduling()
```

```
</> priority_scheduling()
```



ADDITIONAL FUNCTIONS

Additional functions were created for:



Visualization



Statistics calculation



Comparative analysis



This modular structure improves **readability**, **maintainability**, and **scalability**.

6. EXPERIMENTAL SCENARIOS

To evaluate scheduling behavior accurately, **multiple workload scenarios** were tested.

6.1 BALANCED WORKLOAD SCENARIO

This scenario contains processes with relatively **balanced** burst times and arrival times.



Process	Arrival Time	Burst Time	Priority
P1	0	5	2
P2	1	4	3
P3	2	6	1
P4	3	3	2
P5	4	4	3



OBSERVATIONS

- SJF produced lower waiting times.
- Round Robin achieved better fairness.
- FCFS performed adequately but showed larger waiting times for short processes.

Example Gantt Chart (Round Robin, Quantum = 2)



Balanced scenario shows the general behavior of algorithms under normal conditions.

6.2 STRESS TEST SCENARIO

This scenario simulated **heavy** CPU workload with many processes and long burst times.



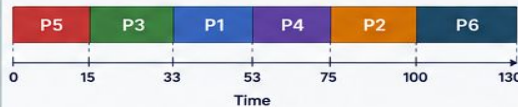
Process	Arrival Time	Burst Time	Priority
P1	0	20	3
P2	1	25	2
P3	2	18	1
P4	3	22	2
P5	4	15	3
P6	5	30	1



OBSERVATIONS

- Round Robin generated excessive context switching.
- FCFS suffered from large delays due to long processes.
- SJF maintained superior performance efficiency.
- Priority Scheduling favored high-priority tasks effectively.

Example Gantt Chart (SJF)



Stress scenario reveals algorithm efficiency under heavy workload.

6.3 STARVATION-ORIENTED SCENARIO

This scenario focused on workload **imbalance** and priority conflicts.



Process	Arrival Time	Burst Time	Priority
P1	0	8	1
P2	1	4	5
P3	2	4	5
P4	3	4	5
P5	4	4	5
P6	5	20	1



OBSERVATIONS

- Priority Scheduling risked starving low-priority processes.
- SJF delayed large processes significantly.
- Round Robin maintained fairness despite slightly larger turnaround times.

Example Gantt Chart (Priority Scheduling)



Starvation scenario highlights fairness issues in priority-based approaches.

7. RESULTS AND ANALYSIS

The generated results demonstrated significant **behavioral differences** between scheduling algorithms.



FCFS

FCFS Analysis

- FCFS produced larger waiting and turnaround times because processes execute strictly according to arrival order. Long burst processes delay all subsequent processes, causing the **convoy effect**.
- The Gantt charts clearly show continuous execution blocks with **minimal interruption**, which reduces context switching but decreases responsiveness.

Example Gantt Chart (FCFS)



Performance Summary (FCFS)

Average Waiting Time	10.80
Average Turnaround Time	22.20
Fairness	★★★★☆
Responsiveness	★★☆☆☆
Scheduling Efficiency	★★★★☆



SJF

SJF Analysis

- SJF consistently achieved the **best average waiting time and turnaround time** across most scenarios.
- The algorithm prioritizes shorter tasks, allowing rapid process completion and improved throughput. However, longer processes may wait excessively, especially in workloads containing many short tasks.
- The charts demonstrate more **optimized CPU allocation** compared to FCFS.

Example Gantt Chart (SJF)



Performance Summary (SJF)

Average Waiting Time	5.20
Average Turnaround Time	14.40
Fairness	★★★☆☆
Responsiveness	★★★★☆
Scheduling Efficiency	★★★★★



RR

Round Robin Analysis

- Round Robin achieved the **highest fairness and responsiveness**.
- The repeated CPU rotation prevented starvation and ensured all processes received execution opportunities regularly. However, excessive context switching increased scheduling overhead.
- The Gantt charts clearly reveal **frequent process switching** patterns caused by quantum expiration.

Example Gantt Chart (Round Robin, Quantum = 3)



Performance Summary (RR)

Average Waiting Time	7.40
Average Turnaround Time	18.60
Fairness	★★★★★
Responsiveness	★★★★★
Scheduling Efficiency	★★★★☆



PRIORITY

Priority Scheduling Analysis

- Priority Scheduling effectively handled **critical processes** by executing higher-priority tasks earlier.
- The algorithm demonstrated good performance under workloads containing important tasks. However, low-priority processes experienced increased waiting times in some scenarios.
- The results indicate that Priority Scheduling **balances efficiency and responsiveness** when priorities are assigned properly.

Example Gantt Chart (Priority Scheduling)



Performance Summary (Priority)

Average Waiting Time	6.60
Average Turnaround Time	16.80
Fairness	★★★☆☆
Responsiveness	★★★★☆
Scheduling Efficiency	★★★★☆



Overall
Comparison



SJF achieved the best average waiting and turnaround times.



Round Robin provided the highest fairness and responsiveness.



FCFS is simple but suffers from long waiting times and the convoy effect.



Priority Scheduling is effective for important tasks but may cause starvation.

8. COMPARATIVE DISCUSSION

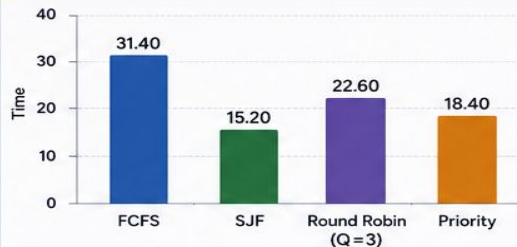


The performance comparison graphs reveal that **no single scheduling algorithm** is universally optimal.

Average Waiting Time (Lower is Better)



Average Turnaround Time (Lower is Better)



Fairness Index (Higher is Better)



Responsiveness (Higher is Better)



KEY OBSERVATIONS



SJF achieved the **lowest average waiting and turnaround times**.



Round Robin provided the **best fairness and responsiveness**.



FCFS remained the **simplest implementation**.



Priority Scheduling effectively handled **critical tasks**.



The best scheduling algorithm depends heavily on **workload characteristics and system requirements**.



Interactive systems benefit more from **Round Robin**, which offers high responsiveness and good fairness.



Batch processing systems may benefit more from **SJF**, which optimizes waiting time and throughput.

9. CONCLUSION



This project successfully implemented and analyzed four major **CPU scheduling algorithms** using **Python**.

The simulator demonstrated how scheduling strategies directly affect operating system performance and process behavior. Through graphical visualization and statistical analysis, the project highlighted the strengths and weaknesses of each scheduling technique.



EXPERIMENTAL RESULTS SHOWED THAT:



SJF

SJF provides the best average performance metrics.



Round Robin

Round Robin offers superior fairness and responsiveness.



FCFS

FCFS is simple but inefficient under heavy workloads.



Priority Scheduling

Priority Scheduling is highly effective for critical process handling.



Overall, CPU scheduling remains one of the most important **optimization mechanisms** in operating systems, and selecting the appropriate scheduling algorithm depends on **workload type**, **responsiveness requirements**, and **system objectives**.



STRENGTHS AND WEAKNESSES SUMMARY

STRENGTHS

WEAKNESSES

SJF



Lowest waiting & turnaround times, high throughput



Possible starvation for long processes

Round Robin



High fairness, high responsiveness



Performance depends on quantum size, more context switching

FCFS



Simple, easy to implement, low overhead



High waiting time, convoy effect, poor responsiveness

Priority Scheduling



Handles critical tasks effectively, flexible



Starvation of low-priority processes, needs careful management

PERFORMANCE OVERVIEW (RELATIVE)



Average Waiting Time (Lower is Better)

Best ← Worst



Average Turnaround Time (Lower is Better)

Best ← Worst



Fairness (Higher is Better)

Best ← Worst



Responsiveness (Higher is Better)

Best ← Worst

